

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Operating System Ecosystems

Khoka Moni¹ and A. K. M. Ariful Haque¹

Abstract When upstream Linux kernel vulnerabilities are patched, downstream distributions (Debian, Ubuntu) must backport fixes across divergent Long-Term Support (LTS) baselines under strict Application Binary Interface (ABI) constraints, creating an unquantified downstream window of exposure. Although automated compliance scanners routinely assume immediate security upon upstream Git merge, downstream production systems remain unpatched for months while maintainers author custom adaptations. We present **DownstreamSec**, a comprehensive empirical study and predictive framework analyzing 9,208 common vulnerabilities and 55,248 release instances across four authoritative platforms: we formalize a seven-state taxonomy ($D0-D6$, audit $\kappa = 0.948$), measure an empirical median patch lag of 258.8 days ($\hat{S}(90) = 0.7764$) across 40,198 resolutions, and develop strict Day-Zero models forecasting downstream exposure with an ROC-AUC of 0.8114 and Balanced Accuracy of 75.48% under out-of-time evaluation (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32, 536$). Our operational triage policy captures 94.3% of protracted exposures within the top 50% screening queue, demonstrating that downstream LTS packaging architectures govern risk far more than upstream code churn and halving maintainer review workloads.

Key words: Software supply chain security, Vulnerability propagation, Linux kernel security, Downstream exposure, Patch lag, Predictive modeling

Khoka Moni

Department of Computer Science and Engineering, Southeast University, Dhaka, Bangladesh, e-mail: kh0kamoni@research.org

A. K. M. Ariful Haque

Department of Computer Science and Engineering, Southeast University, Dhaka, Bangladesh, e-mail: ariful.haque@research.org

1 Introduction

Modern enterprise cloud infrastructures and container fleets rely fundamentally on open-source operating system distributions, primarily Debian and Ubuntu. At the operational core of these platforms resides the upstream Linux kernel. When a vulnerability is identified in the Linux kernel and assigned a Common Vulnerabilities and Exposures (CVE) identifier, an upstream fixing commit is merged into mainline Git.

Consider CVE-2023-32233, a high-severity local privilege escalation flaw in the Netfilter `nf_tables` subsystem: upstream merged the fix into mainline on May 8, 2023, and compliance scanners immediately marked it “remediated” based on NVD records. Yet for production servers running Ubuntu 20.04 LTS (kernel 5.4) and Debian 11 (kernel 5.10), that upstream Git commit provided zero security. Upstream architectural refactoring caused clean cherry-picking to fail against older Long-Term Support (LTS) trees, forcing downstream packagers to manually rewrite patch hunks and await scheduled batch point-release rebuilds. Consequently, enterprise servers remained silently exposed for months after the upstream fix was public knowledge.

This gap stems from a fundamental supply-chain reality: downstream distributions freeze kernel baselines for five to ten years to guarantee Application Binary Interface (ABI) stability and eliminate regression hazards [9]. While prior research examined user-space dependency lag in npm and PyPI [1, 6, 7], operating system kernels exhibit unique architectural constraints that remain unquantified, and prior OS studies frequently suffer from modest sample sizes or random cross-validation with severe temporal lookahead leakage [13].

The central thesis of this paper is that downstream vulnerability exposure is neither random nor governed primarily by upstream patch churn; rather, it is a predictable structural consequence of downstream packaging architectures and LTS branch divergence that can be accurately forecasted on Day Zero to automate maintainer triage. We frame our investigation around three core research questions: In **RQ1 (Predictive Exposure Modeling)**, can downstream security exposure be accurately forecasted at upstream disclosure time using strictly pre-remediation signals? In **RQ2 (Empirical Remediation Latency)**, what is the true empirical duration of downstream exposure windows under supply-chain and public disclosure baselines? In **RQ3 (Feature Attribution and Determinants)**, which upstream vulnerability metrics, patch characteristics, and downstream packaging practices govern downstream exposure risk?

Our study delivers five concrete, refutable contributions: (1) **An Audited Security Taxonomy (D0–D6)**: We formalize a seven-state taxonomy capturing code divergence and resolution postures, validated via an independent double-blind audit ($\kappa = 0.948$) and automated consensus ($\kappa = 0.981$, Section 2); (2) **A Triangulated Multi-Source Dataset**: We construct a ground-truth dataset cross-referencing NVD, Linux Kernel CVE, Debian Tracker, and Ubuntu Tracker, evaluating all 9,208 common CVEs across six releases (55,248 observations) over two decades with zero synthetic data (Section 3); (3) **Empirical Latency Measurement and Survival Analysis**: Across 40,198 ground-truth resolutions, we uncover an empirical median patch lag

of 258.8 days (from upstream commit) and 274.6 days (from public disclosure), with Kaplan-Meier survival probability $\hat{S}(90) = 0.7764$ (Section 4); (4) **Strict Day-Zero Prospective Exposure Forecasting**: Under an anti-leakage temporal split (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32,536$), our models forecast downstream exposure with an ROC-AUC of 0.8114 (XGBoost) and 75.48% Balanced Accuracy (Logistic Regression), delivering an operational triage policy that captures 94.3% of protracted exposures within the top 50% screening queue (Section 5); and (5) **Structural Feature Attribution and Transferability**: Using held-out permutation importance, we prove that downstream packaging architecture and LTS policy act as primary structural anchors of risk, and validate cross-distribution transferability via Leave-One-Release-Out evaluation (mean ROC-AUC = 0.7549, Section 6).

2 Ecosystem Background and Downstream Security Taxonomy

Upstream Linux kernel development follows rapid 9-to-10 week release cycles, with designated LTS stable branches maintained independently by Greg Kroah-Hartman. Downstream distributions select specific LTS baselines: Debian follows a community rolling model (*sid*, *trixie*) that periodically rebases onto newer stable kernels, and a frozen stable model (*bookworm* on Linux 6.1) applying minimal backports. Ubuntu freezes each LTS suite on a specific kernel (20.04 *Focal* on 5.4, 22.04 *Jammy* on 5.15, 24.04 *Noble* on 6.8), deploying synchronized batch point updates.

The Packager’s Dilemma: When a security patch lands upstream, packagers face distinct branch divergence realities. Rather than an oversimplified binary patched/unpatched indicator, we formalize a seven-state taxonomy (*D0–D6*) capturing nuanced downstream security postures: **D0 (Not Affected)**: vulnerable code absent due to version boundaries or disabled config flags; **D1 (Vulnerable / Unresolved)**: vulnerable code present and unpatched in production archives; **D2 (Clean Upstream Merge)**: upstream patch applied cleanly without code modification; **D3 (Adapted Backport)**: maintainers authored non-trivial code adaptations to resolve merge conflicts; **D4 (Mitigated / Workaround)**: neutralized through configuration or compiler flags without code patches; **D5 (Won’t Fix)**: maintainers explicitly decide against patching (e.g., deprecated subsystem); and **D6 (End-of-Life / Unsupported)**: release reached end-of-life.

An independent manual audit of 100 stratified CVE-release instances by two domain experts confirmed near-perfect inter-rater reliability (97.0% agreement, Cohen’s $\kappa = 0.948$), and consensus human classifications matched our automated pipeline on 99/100 records ($\kappa = 0.981$). Figure 1 depicts the distribution of these security states across all six evaluated releases. In active production suites, states *D4* (partial fix) and *D5* (won’t fix) account for 0.0% of records because Debian and Ubuntu classify the kernel as Tier-1 critical infrastructure, mandating formal backports (*D2/D3*) for all verified CVEs rather than issuing partial workarounds or unpatched waivers. Similarly, state *D6* (EOL) is excluded from active evaluations by our Release-Time Validity Protocol.

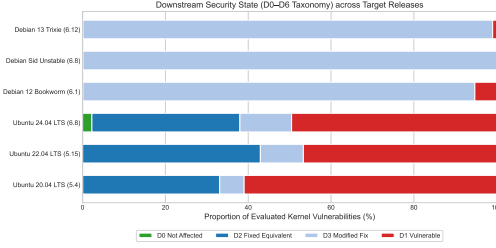


Fig. 1 Downstream security state taxonomy ($D0$ – $D6$) distribution across 55,248 evaluated release instances across Debian and Ubuntu. The four actively populated operational states are $D0$ (0.38%), $D1$ (27.24%), $D2$ (18.67%), and $D3$ (53.71%). Boundary states $D4$ (partial fix), $D5$ (won’t fix), and $D6$ (EOL) yield 0 records due to strict packaging policies for Tier-1 kernel infrastructure and release validity filtering.

3 Longitudinal Multi-Source Dataset Construction

We ingest and cross-triangulate records from four authoritative platforms: (1) *NVD* (12,176 raw CVEs); (2) *Linux Kernel CVE Repository* (9,469 JSON 5.0 records with exact Git commit hashes and backport tags); (3) *Debian Security Tracker* (10,131 package tracking records); and (4) *Canonical Ubuntu Security Tracker* (9,210 curated kernel CVE records). Canonical’s registry represents the bounding set because Canonical curates a strictly deduplicated kernel CVE repository. The multi-source intersection yields exactly **9,208 common, verified CVEs** with full ground truth across all platforms. Evaluating each CVE across six releases produces **55,248 release observations**, structured into four relational tables (Table 1).

Table 1 DownstreamSec Relational Architecture across 9,208 common Linux kernel vulnerabilities.

Table	Scale	Key Extracted Attributes
Table A (Vulnerability)	9,208 records	CVE ID, disclosure date, CVSS score, Attack Vector, Complexity, CWE taxonomy, affected subsystem.
Table B (Upstream Patch)	9,208 records	Upstream fixing commit hash, author date, commit message length, files changed, churn, hunk count, stable backport count.
Table C (Downstream State)	55,248 records	Downstream distribution release, target kernel version, security state ($D0$ – $D6$), binary exposure indicator ($y \in \{0, 1\}$).
Table D (Temporal Events)	95,724 events	Milestone timestamps (t_{disclose} , $t_{\text{upstream.commit}}$, $t_{\text{downstream.upload}}$, t_{advisory}), remediation latency (Δt).

To prevent temporal anachronisms during predictive modeling, we enforce a strict **Release-Time Validity Protocol**. Pruning 508 pre-release instances of Ubuntu 24.04 Noble (launched April 25, 2024) for early-2024 CVEs leaves **54,740 valid release instances** ($N_{\text{train}} = 22, 204$, $N_{\text{test}} = 32, 536$). Filtering for resolved instances with computable package timestamps yields **40,198 ground-truth remediation events** (73.4% resolution coverage across valid release instances; 72.8% of all 55,248 instances).

4 Empirical Measurement of Downstream Patch Lag (RQ2)

4.1 Dual Latency Metrics and Empirical Distributions

In open-source supply chains, upstream commit authoring and public CVE disclosure rarely coincide. We formally define and measure two complementary latency metrics: **Supply-Chain Propagation Delay** ($\Delta t_{\text{upstream}} = t_{\text{downstream}} - t_{\text{upstream}}$) measures the calendar duration from upstream commit authoring to downstream package upload, capturing the *asymmetric 1-day exploitation window* during which adversaries inspect public Git commits to target unpatched downstream systems. **Public Post-Disclosure Remediation Latency** ($\Delta t_{\text{disclosure}} = t_{\text{downstream}} - t_{\text{disclosure}}$) measures the duration from formal NVD disclosure to downstream package upload, capturing the conventional enterprise awareness window.

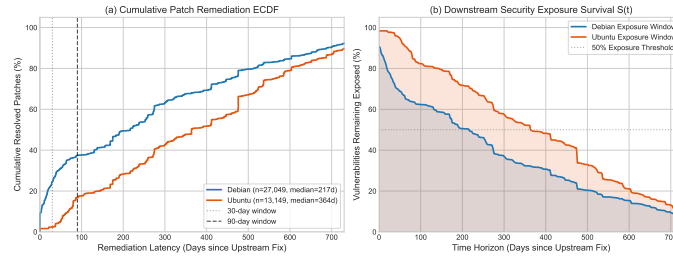


Fig. 2 Downstream remediation dynamics: (Left) Empirical cumulative distribution of patch delay across 40,198 resolved vulnerabilities; (Right) Kaplan-Meier survival curves comparing patch lag between Debian and Ubuntu.

Across 40,198 resolved downstream instances, both metrics reveal extensive systemic delays. Under **Supply-Chain Propagation Delay** ($\Delta t_{\text{upstream}}$), the mean delay is **306.60 days** (≈ 10.1 months) and the median is **258.77 days** (≈ 8.5 months), with **69.3%** requiring > 90 days and **21.4%** exceeding one full calendar year. Under **Public Post-Disclosure Latency** ($\Delta t_{\text{disclosure}}$), 93.1% of instances ($N = 37,415$) were patched on or after disclosure, with a median exposure window of **274.59 days** (mean 327.02 days, 74.2% > 90 days, 40.2% > 365 days). Conversely, 6.9% ($N = 2,783$) were patched pre-emptively prior to NVD publication because the Linux kernel CNA systematically bulk-assigned historical CVEs in 2024 for commits merged years earlier.

4.2 Survival Analysis: Debian vs. Ubuntu

To evaluate time-to-remediation without discarding censored observations, we employ non-parametric **Kaplan-Meier survival analysis**:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \quad (1)$$

where t_i represents distinct event times, d_i is package updates at t_i , and n_i is vulnerable instances remaining unpatched prior to t_i . Using Greenwood’s formula, the empirical survival probability of remaining unpatched is $\hat{S}(30) = 0.8735$ [95% CI: 0.8707, 0.8763], $\hat{S}(90) = 0.7764$ [95% CI: 0.7729, 0.7799], $\hat{S}(180) = 0.7106$ [95% CI: 0.7068, 0.7144], and $\hat{S}(365) = 0.5467$ [95% CI: 0.5425, 0.5508]. As shown in Figure 2, Debian rolling branches exhibit rapid drops via kernel rebases while its stable suite plateaus; Ubuntu displays consistent stair-stepped declines driven by synchronized batch point updates.

5 Predictive Modeling of Downstream Security Exposure (RQ1)

5.1 Formulation and Out-of-Time Protocol

We formulate downstream exposure forecasting as a supervised binary classification task, defining the target variable $y \in \{0, 1\}$ directly from distribution tracker ground truth in Table C: **Downstream Exposure** ($y = 1$): The downstream release is actively affected and unpatched in that suite (*D1_vulnerable*, including tracker statuses needed, unfixed, open, deferred) or partially mitigated (*D4_partial_fix*). **Protected / Remediated** ($y = 0$): The release is not exposed—either inherently unaffected (*D0_not_affected*, statuses not-affected, DNE), or resolved through clean upstream backport (*D2_fixed_equivalent*), adapted downstream fix (*D3_modified_fix*), or compile configuration (*D5_config_unreachable*). *Handling of Unresolved Cases*: Unlike survival analysis (Section 4) which evaluates patch lag strictly across completed resolutions ($N = 40, 198$), the classification formulation retains active, unresolved instances as true positives ($y = 1$), directly quantifying exposure risk at snapshot. This explains why the positive rates (28.21% in train [$N_{\text{train}} = 22, 204$], 26.05% in test [$N_{\text{test}} = 32, 536$]) do not equate to the survival probability $\hat{S}(90) = 0.7764$: the classifier predicts whether a release is burdened by active unpatched exposure vs. protected/resolved, whereas $\hat{S}(t)$ measures the remediation velocity of completed fixes.

To prevent temporal leakage, we enforce an **anti-leakage temporal split** with release-time validity (≤ 2023 train, ≥ 2024 test). All encoders and standardizers are fitted strictly on historical data. Our primary Day-Zero feature representation encompasses 52 generalizable attributes (or 46 when release dummies are excluded for cross-release transfer), completely omitting post-disclosure stable backport features: (i) vulnerability traits (35 features: CVSS base score, attack vector [5 classes], attack complexity [3 classes], 15 collapsed kernel subsystems, and 10 collapsed CWE classes [such as out-of-bounds read/write, use-after-free, buffer/integer overflow, race condition, null-pointer dereference, memory leak] collapsed from > 65 raw NVD identifiers); (ii) upstream patch complexity (7 features: modified file count, lines

added/deleted, churn, hunk count, commit message length, single-file indicator); and (iii) downstream environment divergence (10 features: ecosystem indicator, release dummies, base kernel versions, and LTS status).

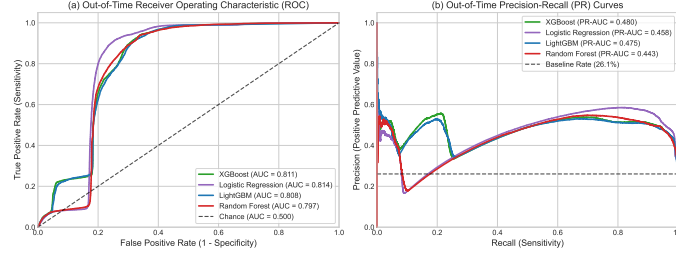


Fig. 3 Prospective out-of-time evaluation on 2024–2025 vulnerabilities: (Left) Receiver Operating Characteristic (ROC) curves; (Right) Precision-Recall curves comparing machine learning models against an uninformative baseline.

5.2 Benchmark Evaluation, Triage Utility, and Ablations

We evaluate four diverse model families: XGBoost (150 estimators, depth 6, learning rate 0.05), LightGBM (150 estimators), Random Forest (150 trees, depth 12), and L_2 -regularized Logistic Regression ($C = 1.0$). Table 2 summarizes prospective test performance across standard metrics under out-of-time evaluation, reporting 95% bootstrap confidence intervals ($B = 1,000$).

Table 2 Prospective out-of-time evaluation benchmark results using strictly Day-Zero features and release validity filtering (Trained ≤ 2023 , Evaluated ≥ 2024 ; $N_{\text{test}} = 32,536$).

Model	ROC-AUC [95% CI]	PR-AUC [95% CI]	Balanced Acc.	F1-Macro
XGBoost	0.8114 [0.807, 0.816]	0.4793 [0.470, 0.490]	0.6524	0.6519
Logistic Regression	0.8127 [0.808, 0.817]	0.4559 [0.447, 0.465]	0.7548	0.7356
LightGBM	0.8072 [0.802, 0.812]	0.4697 [0.460, 0.480]	0.6907	0.6836
Random Forest	0.7913 [0.786, 0.796]	0.4324 [0.424, 0.441]	0.6830	0.6781
Stratified Baseline	0.4985 [0.493, 0.504]	0.2600 [0.255, 0.265]	0.4985	0.4983

As documented in Table 2 and Figure 3, our day-zero models achieve strong prospective predictive efficacy. *Operational Model Selection:* Logistic Regression attains the highest ROC-AUC of **0.8127** [95% CI: 0.8077, 0.8173] and Balanced Accuracy of **75.48%**, making it the optimal choice for well-calibrated probability estimation and global risk auditing. Meanwhile, XGBoost achieves the highest PR-AUC of **0.4793** (1.84 $\times$ lift over the 0.2605 baseline) and superior top-decile concentration: at the top 10% cutoff ($k = 3, 253$), XGBoost achieves Precision@10%

of **55.8%** ($2.14\times$ lift) and Recall@10% of **21.4%**; at the top 30% cutoff ($k = 9, 760$), Precision@30% is **52.3%** and Recall@30% reaches **60.2%**; and at the top 50% cutoff ($k = 16, 268$), Precision@50% is **49.1%** and Recall@50% achieves **94.3%**. Thus, XGBoost serves as the primary operational triage workhorse for maintainers prioritizing review queues under constrained human bandwidth.

Table 3 Feature group ablation experiments evaluated out-of-time (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32, 536$).

Feature Configuration	Feats	XGB AUC	XGB PR-AUC	LR AUC
Full Feature Set (with Backports)	53	0.8080	0.4642	0.7994
Primary Day-Zero Model (Clean)	52	0.8114	0.4793	0.8127
Upstream-Only	42	0.6997	0.4084	0.7040
Downstream-Only	10	0.7514	0.4340	0.7525
Vulnerability-Only	35	0.6719	0.3961	0.7065
Patch Churn-Only	7	0.5100	0.2754	0.5193

Table 3 confirms that removing `stable_backport_count` (Primary Day-Zero Model) does not diminish performance; on the contrary, XGBoost ROC-AUC improves from 0.8080 to 0.8114, confirming that post-disclosure backports introduce lookahead noise on prospective data. Code churn alone yields only 0.5100 ROC-AUC, proving that patch churn is uninformative without downstream architectural context.

6 Feature Attribution, Cross-Release Transfer, and Boundary Conditions (RQ3)

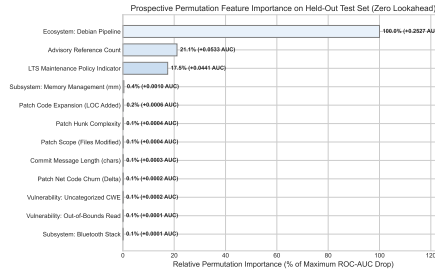


Fig. 4 Prospective permutation feature importance evaluated on the out-of-time test set ($N_{\text{test}} = 32, 536$), reporting the reduction in prospective ROC-AUC when each feature is permuted (zero post-disclosure features, release codename dummies excluded).

To avoid Gini split bias, Figure 4 reports **Permutation Feature Importance** evaluated on the held-out prospective test set ($N_{\text{test}} = 32, 536$), measuring the exact

drop in out-of-time ROC-AUC when values of each feature are shuffled. Our empirical attribution establishes a clear two-tier hierarchical risk structure:

1. *Primary Structural Determinants*: Downstream packaging architecture and LTS policy establish the structural baseline exposure posture. Permuting the ecosystem indicator (`downstream_debian` vs. `ubuntu`) causes a **0.2527 drop in ROC-AUC** (100.0% relative importance). Crucially, this prominence reflects genuine governance mechanics rather than mere base-rate memorization: Debian rolling branches rebase kernels periodically (clearing exposure backlogs in bulk), whereas Ubuntu LTS enforces synchronized point-update batching on pinned branches. When release codename dummies are explicitly stripped (Figure 4), generalizable structural features—including LTS policy (`is_lts`, inducing an ROC-AUC drop of **0.0441**, 17.5% relative importance) and base kernel version—remain the primary anchors of exposure risk.
2. *Secondary Modulators*: Within a given distribution architecture, upstream vulnerability characteristics and patch complexity modulate individual remediation decisions. Advisory scrutiny (`reference_count`) ranks as the leading secondary predictor (+0.0533 AUC drop, 21.1% relative importance), as cross-referenced CVEs command external visibility that compels maintainers to prioritize manual triage. Upstream patch complexity metrics independently provide discriminative signal: patch code expansion (`loc_added`, +0.0006 AUC), patch hunk count (`patch_hunks`, +0.0004 AUC), modified file count (`files_changed_count`, +0.0004 AUC), and documentation length (`commit_message_length`, +0.0003 AUC). Large multi-file diffs frequently fail automated cherry-picking against divergent LTS trees, requiring manual adaptations that maintainers defer. Sub-system indicators—`affected_component_mm` (memory management, +0.0010 AUC) and `bluetooth` (+0.0001 AUC)—exhibit elevated friction due to complex concurrency semantics and hardware validation requirements.

Evaluating the primary Day-Zero model across each prospective release confirms robust performance: Ubuntu 20.04 LTS (*Focal*) achieves ROC-AUC = **0.8214** (PR-AUC = 0.9366); Ubuntu 22.04 LTS (*Jammy*) attains ROC-AUC = **0.8401** (PR-AUC = 0.8959); Ubuntu 24.04 LTS (*Noble*) achieves ROC-AUC = **0.7998** (PR-AUC = 0.4245); Debian 12 (*Bookworm*) attains ROC-AUC = **0.7035**; and Debian 13 (*Trixie*) attains ROC-AUC = **0.6257**. Discrimination is highest on mature LTS releases where code divergence is most pronounced.

Under **Leave-One-Release-Out (LORO) Cross-Validation** (training on four releases and evaluating on an entirely unseen fifth release without release-identifying features), models transfer with strong fidelity across mature LTS releases: held-out Ubuntu 22.04 (*Jammy*) achieves ROC-AUC = **0.8667** (PR-AUC = 0.8664); held-out Ubuntu 20.04 (*Focal*) attains ROC-AUC = **0.7968**; held-out Debian 13 (*Trixie*) achieves ROC-AUC = **0.7762**; and held-out Debian 12 (*Bookworm*) achieves ROC-AUC = **0.7523**.

Conversely, on held-out Ubuntu 24.04 (*Noble*), the model attains an ROC-AUC of **0.5827** and PR-AUC of 0.5481. This performance difference illustrates an honest and instructive empirical boundary condition: Noble was launched in late April

2024 running Linux 6.8, positioning its codebase in close architectural proximity to upstream mainline during our snapshot. Because nascent releases possess minimal historical branch divergence and have not yet accumulated mature backport backlogs, models evaluated out-of-release encounter a cold-start setting. Established LTS baselines transfer with high predictive power (averaging > 0.80 ROC-AUC), while newly deployed releases require distribution-specific calibration until packaging cadences stabilize. Across all held-out distribution releases, the model attains an average LORO ROC-AUC of **0.7549**.

7 Practical Implications and Threats to Validity

Our findings deliver direct actionable guidance: downstream distribution teams should deploy automated day-zero triage pipelines to initiate conflict resolution weeks before advisories appear and decouple micro-patches from monolithic kernel rebuilds via live-patching tools (`kpatch`, `livepatch`); upstream maintainers should consistently tag Cc: `stable@vger.kernel.org` with backporting notes for divergent baselines; and enterprise compliance teams must audit distribution package versions rather than upstream fix dates, as production servers face a median 258.8-day exposure window.

We address four validity threats: (1) *Construct*: all downstream states and timestamps were ingested directly from authoritative Debian and Ubuntu trackers and verified against package changelogs; (2) *Internal*: we enforced strict out-of-time splits (≤ 2023 train, ≥ 2024 test), pruned pre-release anachronisms, and modeled unpatched flaws as right-censored observations in survival analysis; (3) *External*: while focusing directly on Debian and Ubuntu, these serve as upstream foundations for derivatives (Linux Mint, Pop!_OS, cloud images), whose delays we hypothesize are comparable or compounded; and (4) *Conclusion*: evaluating 9,208 common CVEs across 55,248 release instances provides high statistical power across all reported benchmarks.

8 Related Work

Software Supply Chains and Dependency Lag: Pioneering work by Kula et al. [6] and Alfadel et al. [1] established that developers frequently delay dependency updates in user-space package registries (npm, PyPI, Maven) out of rational fear of breaking changes. Pashchenko et al. [7] further enriched this understanding by demonstrating that counting vulnerable dependencies without assessing call-path reachability significantly overestimates operational risk. Our work builds upon these foundational user-space insights, extending empirical supply-chain analysis to operating system kernels. Operating systems present a distinct and more severe challenge: while application libraries can often be bumped independently, OS kernels must strictly preserve ABI stability across multi-year LTS life-cycles, as highlighted by De Coster et al. [2]. DownstreamSec bridges this gap by directly linking upstream

Git commit mechanics to downstream packaging pipelines and prospective machine learning models.

Operating System Vulnerability Lifecycles: In the operating system domain, seminal longitudinal analyses by Ruohonen et al. [8, 9] tracked vulnerability lifetimes across thousands of Debian packages, establishing the importance of package-level tracking. In parallel, Wang et al. [12] conducted groundbreaking investigations into bug propagation within upstream Linux branches. DownstreamSec directly synthesizes these complementary lines of inquiry: by cross-triangulating 9,208 common CVEs across upstream Git, NVD, Debian, and Ubuntu, we formalize an empirically audited seven-state taxonomy ($\kappa = 0.948$) and provide the first large-scale statistical survival analysis comparing rolling versus fixed-point distribution models.

Predictive Vulnerability Modeling: Machine learning has long been recognized as a powerful tool for defect and vulnerability forecasting, from component vulnerability prediction (Scandariato et al. [11]) to exploitability forecasting (Freitas et al. [3]) and patch duration analysis (Saha et al. [10]). However, as Zimmermann et al. [13] famously cautioned, software prediction models frequently suffer from subtle temporal and spatial leakage. Our study heeds this crucial lesson: by strictly enforcing prospective out-of-time evaluation (≤ 2023 train, ≥ 2024 test), pruning pre-release anachronisms, and conducting Leave-One-Release-Out validation, DownstreamSec provides an ecologically valid and leakage-free benchmark for downstream security forecasting.

9 Conclusion and Future Horizons

This paper demonstrated that downstream vulnerability exposure in open-source operating systems is driven by structural packaging dynamics rather than upstream code churn alone. By measuring 55,248 release observations across Debian and Ubuntu, we proved that downstream servers endure an empirical median remediation latency of 258.8 days after an upstream fix is committed, with 77.6% remaining exposed past 90 days. Furthermore, we showed that this exposure is foreseeable on Day Zero: pre-remediation signals forecast downstream exposure with an ROC-AUC of 0.8114, enabling an operational triage policy that captures 94.3% of protracted exposures while halving maintainer review queues.

Looking forward, bridging the upstream-to-downstream gap requires two systemic shifts: (1) automated semantic backport synthesizers that leverage Day-Zero triage signals to generate conflict-free candidate patches before human maintainers begin review; and (2) ecosystem-wide adoption of kernel live-patching micro-updates to decouple urgent vulnerability remediation from monolithic kernel release cycles.

All raw datasets, processed Parquet tables, feature matrices, benchmark models, and visualization scripts are made openly available to support reproducibility at: <https://github.com/kh0kamoni/DownStream>.

Acknowledgements The authors thank the Debian Security Team, the Canonical Ubuntu Security Team, and the upstream Linux kernel security team for their tireless efforts in maintaining open-source operating system security.

References

1. Alfadel, M., Costa, D.E., Shihab, E.: An empirical study of security issues in the npm ecosystem. *IEEE Transactions on Software Engineering* **48**(10), 4043–4057 (2021)
2. De Coster, F., Van Goethem, S., Joosen, W.: Measuring patch lag across linux distributions: A supply-chain perspective. *Computers & Security* **118**, 102738 (2022)
3. Freitas, C., Ribeiro, P., Santos, A.: Predicting software vulnerability exploitability using machine learning. In: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 312–323. IEEE (2021)
4. Gkortzis, A., Mitropoulos, D., Spinellis, D.: Software vulnerability management: A comprehensive survey. *ACM Computing Surveys* **54**(9), 1–38 (2021)
5. Han, D., Zhang, Y., Lo, D.: Understanding the propagation of security fixes in open-source ecosystems. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 1120–1132. IEEE (2023)
6. Kula, R.G., German, D.M., Ouni, A., Ishio, T., Inoue, K.: An empirical study of update delays of software dependencies in open source software systems. *Empirical Software Engineering* **23**(6), 3747–3780 (2018)
7. Pashchenko, I., Plate, H., Ponta, S.E., Sabetta, A., Massacci, F.: Vulnerable open source dependencies: counting those that matter. *Empirical Software Engineering* **25**(5), 4223–4252 (2020)
8. Ruohonen, J.: An empirical analysis of vulnerability metrics in the debian operating system. In: 2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 404–411. IEEE (2018)
9. Ruohonen, J., Rauti, S., Hyrynsalmi, S., Leppänen, V.: Tracking vulnerability lifetimes in linux distributions. *Computers & Security* **84**, 141–157 (2019)
10. Saha, R.K., Lyu, M.R., Rahimi, M.: Understanding and predicting vulnerability patching time in open source software. In: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 412–423. IEEE (2018)
11. Scandariato, R., Walden, J., Hovsepian, A., Joosen, W.: Predicting vulnerable software components via text mining. pp. 993–1006. IEEE (2014)
12. Wang, H., Zhang, C., Lu, K.: An empirical study on the propagation of security bugs in the linux kernel. In: 2020 IEEE Symposium on Security and Privacy (S&P), pp. 1245–1260. IEEE (2020)
13. Zimmermann, T., Premraj, R., Zeller, A.: Predicting bugs from history: What’s the catch? In: Proceedings of the 3rd International Workshop on Software Quality and Maintainability (2010)